

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Построение и анализ алгоритмов»
ТЕМА: ПОИСК НАБОРА ПОДСТРОК В СТРОКЕ.

Студент гр. 4345

Наумов М. В.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

Задание

Используя реализацию точного множественного поиска, решите задачу точного поиска для одного образца с джокером.

В шаблоне встречается специальный символ, именуемый джокером (wild card), который "совпадает" с любым символом. По заданному содержащему шаблоны образцу P необходимо найти все вхождения P в текст T .

Например, образец $ab??c?ab??c?$ с джокером $??$ встречается дважды в тексте $xabvssc bababcsaxx abvssc bababcsax$.

Символ джокер не входит в алфавит, символы которого используются в T . Каждый джокер соответствует одному символу, а не подстроке неопределённой длины. В шаблон входит хотя бы один символ не джокер, т.е. шаблоны вида $???$ недопустимы. Все строки содержат символы из алфавита $\{A, C, G, T, N\}$.

Вход:

Текст $(T, 1 \leq |T| \leq 1000000, 1 \leq |T| \leq 100000)$

Шаблон $(P, 1 \leq |P| \leq 40, 1 \leq |P| \leq 40)$

Символ джокера

Выход:

Строки с номерами позиций вхождений шаблона (каждая строка содержит только один номер).

Номера должны выводиться в порядке возрастания.

Тестовые данные			^
№ Теста	Входные данные	Выходные данные	
1	ACTANCA A\$\$\$ \$	1	

Также задание было дополнено индивидуальным вариантом 5.1:

На месте джокера может быть любой символ, за исключением заданного.

Описание алгоритма

1. Концептуальные основы алгоритма Ахо-Корасика

Фундаментальная задача алгоритма Ахо-Корасика — эффективный одновременный поиск множества подстрок в заданном тексте. В отличие от алгоритмов, ориентированных на одиночный паттерн, данный метод реализует концепцию детерминированного конечного автомата. Принцип однократного прохода текста достигается за счёт предварительного построения древовидной структуры — Бора (Trie), представляющего собой совокупность всех искомых подстрок (в данной работе — частей шаблона, разделенных джокерами). Вместо того чтобы совершать повторные проверки для каждой части паттерна, алгоритм переключается между состояниями автомата, используя информацию о суффиксах уже обработанных символов.

2. Структура данных и система ссылок: Формирование «карты переходов»

Ключевым инструментом оптимизации в Ахо-Корасике является система суффиксных (link) и терминальных (term_link) ссылок.

- Суффиксные ссылки: Для каждого узла Бора вычисляется ссылка на узел, представляющий максимально возможный собственный суффикс текущей строки, присутствующий в дереве. Это аналог префикс-функции в КМП, расширенный на древовидную структуру. Линейность построения обеспечивается обходом в ширину (BFS), где ссылки для текущего уровня вычисляются на основе уже готовых ссылок родительских узлов.

- Терминальные ссылки (Словарные): Оптимизация поиска, позволяющая за один шаг «прыгать» к ближайшему узлу, являющемуся концом искомой подстроки. Это исключает избыточный перебор всех

суффиксов вручную и гарантирует линейную сложность поиска вхождений.

- Динамика откатов: При отсутствии прямого перехода по символу в Боре, автомат совершает переход по суффиксной ссылке до тех пор, пока не будет найдено соответствие или не будет достигнут корень.

3. Механизм поиска и декомпозиция шаблона с джокерами

В процессе сканирования основного текста указатель автомата движется исключительно слева направо. Обработка джокеров реализована через декомпозицию исходного паттерна P на части (подстроки), не содержащие специальных символов.

- Массив счётчиков: Для каждой позиции текста заводится ячейка в массиве `snt`. При обнаружении вхождения любой части шаблона в тексте вычисляется предполагаемая позиция начала всего паттерна. Соответствующая ячейка в `snt` инкрементируется.
- Верификация и запрещённые символы: После завершения прохода проверяются позиции, набравшие количество вхождений, равное числу частей шаблона. Финальный этап включает проверку на соответствие символов на позициях джокеров: вхождение считается валидным только в том случае, если ни один «джокерный» символ текста не совпадает с заранее заданным запрещённым символом (`bad`). Это позволяет учитывать дополнительные ограничения задачи, сохраняя при этом общую высокую производительность алгоритма.

4. Анализ вычислительной сложности

Временная сложность: $O(|T| + |P| + K)$, где K — общее количество промежуточных вхождений частей шаблона.

Эффективность алгоритма обусловлена использованием детерминированного конечного автомата, что позволяет избежать многократного перечитывания основного текста и осуществлять поиск всех частей шаблона параллельно.

- **Препроцессинг:** Разбиение шаблона, построение Бора и вычисление суффиксных и терминальных ссылок выполняется за $O(|P| * n)$ что с учётом константного размера алфавита ($\{A, C, G, T, N\}$) сводится к $O(|P|)$.
- **Поиск (Сканирование):** Проход по тексту выполняется за $O(|T|)$. Переходы по состояниям автомата амортизированно занимают $O(1)$ на каждый символ текста. Обновление массива счётчиков происходит за время, пропорциональное числу найденных подстрок ($O(K)$).
- **Верификация:** Итоговый проход по массиву `snt` и проверка джокеров на совпадение с запрещённым символом занимает $O(|T| + C * |P|)$, где C — количество потенциальных кандидатов, собравших все части шаблона. На практике C крайне мало.
- **Константные операции:** Извлечение ссылок, инкремент элементов массива по индексу и синхронное логирование выполняются за $O(1)$ и не ухудшают общую асимптотическую сложность.

Сложность по памяти: $O(|T| + |P|)$

Пространственная сложность складывается из двух основных структур:

- **Хранение самого автомата** (узлы Бора, переходы `childs`, суффиксные и терминальные ссылки, списки смещений `out`) требует памяти, строго пропорциональной суммарной длине частей шаблона, то есть $O(|P|)$.

- Для реализации логики поиска паттерна с джокерами используется целочисленный массив `snt` размером с длину текста, что требует $O(|T|)$ дополнительной памяти.
- Очередь для BFS и списки `parts` требуют незначительного объема памяти, укладывающегося в $O(|P|)$.

Механизм работы автомата Ахо-Корасика и процесса декомпозиции

Реализованный алгоритм Ахо-Корасика отказывается от последовательного перебора вариантов в пользу построения детерминированного конечного автомата (Бора). Весь процесс адаптирован для поиска шаблона с джокерами и разделен на три этапа, объединенных логикой накопления частичных совпадений.

1. Декомпозиция шаблона и построение Бора

Перед началом поиска выполняется предварительный анализ шаблона `P`, направленный на выделение «чистых» подстрок (частей), не содержащих джокеры.

Метод декомпозиции: Шаблон разбивается на кортежи (подстрока, смещение), где смещение определяет позицию части относительно начала исходного паттерна.

Формирование структуры: Из выделенных частей строится Бор (Trie). Каждый узел представляет состояние автомата, а терминальные узлы содержат список `out`, в который записываются параметры всех подстрок (длина и относительное смещение), заканчивающихся в данной точке.

2. Формирование системы ссылок (`link` и `term_link`)

Для обеспечения эффективных переходов при несовпадении символов используется механизм двухуровневых ссылок, вычисляемых методом обхода в ширину (BFS):

Суффиксные ссылки (`link`): Аналог префикс-функции в КМП, но для древовидной структуры. Если в текущем состоянии переход по символу

невозможен, автомат мгновенно перемещается в узел, представляющий максимально длинный суффикс текущей строки, уже существующий в Боре.

Терминальные ссылки (`term_link`): Оптимизированный маршрут, который ведет к ближайшему узлу, являющемуся концом какой-либо подстроки. Это позволяет за один шаг обнаруживать «вложенные» вхождения (например, если одна часть шаблона является суффиксом другой).

3. Процедура идентификации и верификация вхождений

Поиск в тексте *T* реализует принцип однократного прохода с использованием массива счётчиков `cnt` для восстановления целостности шаблона:

Линейное сканирование: Указатель перемещается по тексту *T* строго вперед. При каждом символе автомат переходит в новое состояние, используя таблицу дочерних узлов или суффиксные ссылки.

Накопление совпадений: При достижении любого терминального узла (через основные переходы или `term_link`) алгоритм обращается к списку `out`. Для каждого вхождения вычисляется потенциальная точка начала всего шаблона в тексте. В ячейке `cnt[start]` инкрементируется количество найденных частей.

Финальная фильтрация: Позиция считается истинным вхождением, если количество найденных частей совпадает с общим числом подстрок в декомпозиции.

Заметка для отчёта (Учёт запрещённого символа):

В рамках усложнённой задачи, после подтверждения целостности паттерна в массиве `cnt`, выполняется дополнительная проверка позиций, соответствующих джокерам. Если символ текста в позиции джокера совпадает с заданным запрещённым символом (`bad symbol`), данное вхождение аннулируется, обеспечивая строгое соблюдение ограничений алфавита задачи.

Структуры данных и ключевые переменные:

В реализации алгоритма Ахо-Корасика для поиска с джокерами используются следующие ключевые элементы:

- `text` и `pattern` — строковые переменные. В отличие от базовых алгоритмов, `pattern` здесь подвергается предварительной декомпозиции, а `text` сканируется один раз без возвратов указателя.
- `wild` и `bad` — символьные переменные, определяющие логику работы с «джокером» и «запрещённым символом» соответственно. Они используются на этапе разбиения шаблона и на этапе финальной верификации кандидатов.
- `Node (class)` — основная структурная единица Бора. Каждый объект класса содержит:
 - `childs (dict)` — переходы к дочерним узлам по конкретным символам.
 - `link (Node object)` — суффиксная ссылка, указывающая на узел, соответствующий самому длинному суффиксу текущей строки.
 - `term_link (Node object)` — терминальная ссылка, позволяющая мгновенно перемещаться к ближайшему узлу, завершающему какую-либо подстроку.
 - `out (list of tuples)` — список найденных подстрок в данной точке Бора (хранит длину и смещение относительно начала шаблона).
- `parts (list of tuples)` — результат декомпозиции шаблона. Содержит пары (подстрока, смещение). Позволяет алгоритму игнорировать джокеры на этапе работы автомата, превращая одну сложную задачу в поиск множества простых подстрок.
- `tree (Node)` — корень Бора. Служит начальной точкой для любого поиска и «точкой сброса» при отсутствии совпадений (аналог нулевого значения в `pref` для КМП).

- cnt (list / array) — массив счётчиков вхождений, размер которого равен длине текста text. Каждая ячейка cnt[start] аккумулирует количество частей шаблона, которые начинаются в данной позиции. Это позволяет восстановить информацию о структуре шаблона после его фрагментации.
- current и check (Node pointers) — динамические указатели на текущее состояние автомата.
 - current перемещается вниз по дереву или по суффиксным ссылкам в процессе чтения текста.
 - check используется для рекурсивного (через term_link) обхода всех найденных подстрок в текущей позиции текста.
- log (file object) — служебный объект для организации подробной трассировки в файл log.txt. В него записываются этапы разбиения строки, процесс построения ссылок в BFS и пошаговая работа автомата в тексте, что критически важно для отладки сложной логики джокеров.

Тестирование

Результаты тестирования представлены в табл. 1.

Таблица 1 – Результаты тестирования

№ п/п	Входные данные	Выходные данные	Комментарии
1.	ACTANCA A\$\$A \$ G	1 4	Простой пример, решения находятся корректно
2.	ACTANCA A\$\$A\$ \$ T		Вариант с запретом отсеивается
3.	ACTANCA A\$\$A \$ T	4	Варианты с запретом успешно отсеиваются, а иные варианты разрешаются

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: fl0.py

```
from collections import deque

class Node:
    def __init__(self, letter='', parent=None):
        self.letter = letter
        self.parent = parent
        self.childs = {}

        self.link = None
        self.term_link = None

        self.out = []

log = open("log.txt", "w", encoding="utf-8")
log.write("--- Начало работы алгоритма Ахо-Корасика с джокерами ---\n")

text = input().strip()
pattern = input().strip()

wild = input().strip()
bad = input().strip()

log.write(f"Считан текст: '{text}'\n")
log.write(f"Считан шаблон: '{pattern}'\n")
log.write(f"Джокер: '{wild}', Запрещенный символ: '{bad}'\n\n")

parts = []

i = 0

log.write("--- Разбиение шаблона на подстроки ---\n")
while i < len(pattern):
    if pattern[i] == wild:
        i += 1
        continue

    start = i
    cur = []

    while i < len(pattern) and pattern[i] != wild:
        cur.append(pattern[i])
        i += 1

    parts.append(''.join(cur), start)
    log.write(f"Извлечена часть: '{''.join(cur)}' со смещением {start}\n")

tree = Node()

log.write("\n--- Построение Бора (Trie) ---\n")
for word, shift in parts:
    log.write(f"Добавление слова '{word}' (смещение {shift}) в Бор\n")
    current = tree
```

```

for ch in word:

    if ch not in current.childs:
        log.write(f"    Создан новый узел для символа '{ch}'\n")
        current.childs[ch] = Node(ch, current)
    else:
        log.write(f"    Переход в существующий узел '{ch}'\n")

    current = current.childs[ch]

    current.out.append((len(word), shift))
    log.write(f"    Узел '{ch}' помечен как конец подстроки. Добавлено в out:
длина {len(word)}, смещение {shift}\n")

tree.link = tree

queue = deque()

log.write("\n--- Построение суффиксных и терминальных ссылок (BFS) ---\n")
log.write("Инициализация ссылок для детей корня\n")
for child in tree.childs.values():
    child.link = tree
    queue.append(child)

while queue:

    current = queue.popleft()

    for ch, child in current.childs.items():

        queue.append(child)

        link = current.link

        log.write(f"Поиск суффиксной ссылки для узла '{ch}'\n")

        while link != tree and ch not in link.childs:
            link = link.link

        if ch in link.childs:
            child.link = link.childs[ch]
            log.write(f"    Установлена суффиксная ссылка\n")
        else:
            child.link = tree
            log.write(f"    Суффиксов в дереве нет, ссылка ведет в корень\n")

        if child.link.out:
            child.term_link = child.link
            log.write(f"    Установлена терминальная ссылка\n")
        else:
            child.term_link = child.link.term_link

cnt = [0] * len(text)

current = tree

log.write("\n--- Поиск вхождений подстрок в тексте ---\n")
for i in range(len(text)):

    ch = text[i]
    log.write(f"\nИтерация i={i}, текущий символ текста '{ch}'\n")

    while current != tree and ch not in current.childs:
        log.write(f"    Нет перехода по '{ch}'. Прыжок по суффиксной

```

```

ссылке.\n")
    current = current.link

    if ch in current.childs:
        current = current.childs[ch]
        log.write(f"    Успешный переход в узел '{ch}'\n")
    else:
        current = tree
        log.write(f"    Сброс указателя в корень дерева\n")

    check = current

    while check:

        for length, shift in check.out:

            start = i - length + 1 - shift
            log.write(f"    Найдено совпадение! Длина: {length}, Смещение:
{shift}.\n")
            log.write(f"    Возможная стартовая позиция всего шаблона:
{start}\n")

            if 0 <= start and start + len(pattern) <= len(text):
                cnt[start] += 1
                log.write(f"        -> Счетчик совпадений для позиции {start}
увеличен до {cnt[start]}\n")

            check = check.term_link

need = len(parts)

log.write(f"\n--- Финальная проверка кандидатов. Нужно совпадений частей:
{need} ---\n")
for start in range(len(text) - len(pattern) + 1):
    if cnt[start] != need:
        continue

    log.write(f"\nКандидат на позиции {start} собрал все {need} частей.\n")
    ok = True

    for j in range(len(pattern)):
        if pattern[j] == wild:
            if text[start + j] == bad:
                log.write(
                    f"        Ошибка: Джокер совпал с запрещенным символом '{bad}'
на позиции {start + j}. Кандидат отклонен.\n")
                ok = False
                break

    if ok:
        log.write(f"    -> Вхождение шаблона полностью подтверждено! Выводим
позицию: {start + 1}\n")
        print(start + 1)

log.write("\n--- Завершение алгоритма ---\n")
log.close()

```